

Mining Legacy Python Corpora into a Provenance-Tracked Automation Platform with Human-in-the-Loop Delivery

Johnny Morgan
Information Systems, UMBC
yj07538@umbc.edu

Abstract—Organizations accumulate years of Python code across many projects performing similar but distinct work, leaving substantial capability locked in unindexed scripts. We present a layered platform that converts such a corpus into an automated product-delivery system. Layer 1 crawls the filesystem and catalogs every source file and its documentation—location, project, purpose, description—in a central database. Layer 2 detects repeated code blocks performing the same purpose using three tiers of clone detection (normalized-AST hashing, FAISS similarity over structural n-grams, and semantic adjudication). Layer 3 consolidates clone clusters into library functions published to a private PyPI server, with mandatory database provenance linking every function to the source blocks it replaced. Layer 4 validates the libraries by re-executing original workloads and comparing outputs against golden masters; no script retires unvalidated. Layer 5 exposes a generative interface that resolves customer requests to cataloged capabilities, schedules execution, and routes every product through a human reviewer who approves release or rejects for rework. Requests with no matching capability route to data scientists, whose code onboards through the same pipeline—closing the growth loop. The same machinery drives scheduled periodic reporting. The design’s invariants are a single database spine, provenance at every level, validation-gated retirement, and human-gated release.

I. INTRODUCTION

The maintenance reality of long-lived analytical organizations is a filesystem of project directories: hundreds of scripts, written over years, solving overlapping problems in incompatible ways. The cost is threefold—rediscovery is manual, duplication compounds, and every new request restarts near zero. Refactoring wholesale is infeasible; the corpus is too large and its behavior is the only specification that exists.

We treat the corpus itself as the specification. The platform mines it in layers: index, detect duplication, consolidate with provenance, validate against original behavior, then serve requests against the validated catalog. Two properties distinguish the design from generic refactoring tooling: *provenance is never optional* (every library function, job, and delivered product traces to its sources), and *humans gate release* (the generative layer proposes and executes; only a named reviewer releases to a customer).

II. DESIGN INVARIANTS

- 1) **The database is the spine.** Nothing is “known” to the platform unless it is a row. All six layers read and write one catalog.

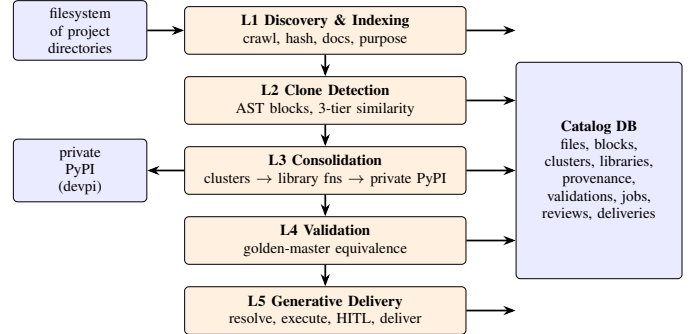


Fig. 1. Layered pipeline over a single catalog database. Each layer writes what the next layer reads; the private PyPI server holds consolidated libraries.

- 2) **Provenance is mandatory.** Library functions record source blocks; jobs record exact library versions; approvals record the reviewer.
- 3) **Nothing retires unvalidated.** An original script is superseded only after Layer 4 proves output equivalence.
- 4) **Humans gate release.** No product reaches a customer without an approval row.

III. LAYERED ARCHITECTURE

Fig. 1 shows the pipeline; all layers converge on the catalog.

A. L1: Corpus Discovery and Indexing

A crawler walks the directory roots, identifying Python sources, notebooks, and documentation. Each file is recorded with path, inferred project, content hash, imports, and entry points. Purpose and description come from docstrings and READMEs where present; otherwise an LLM summarization pass generates them, flagged as machine-generated so provenance of the description itself is preserved. Indexing is idempotent and incremental, keyed on content hash.

B. L2: Code-Block Analysis

Files are parsed to ASTs; candidate blocks (functions, methods, statement spans above a minimum size) are extracted with normalized fingerprints. Similarity runs in three tiers of increasing cost, each only over the prior tier’s survivors: (T1) normalized-AST hashing for exact and renamed clones; (T2) cosine similarity over AST-n-gram vectors in a FAISS index for modified copies; (T3) code-embedding similarity with

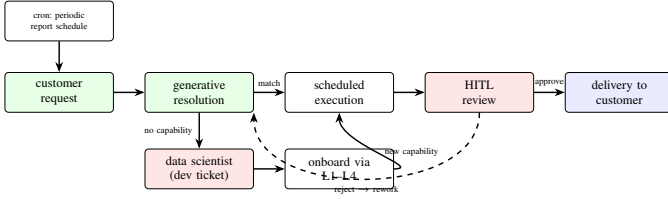


Fig. 2. Request lifecycle. Every product passes the human gate; capability gaps route to development and re-enter through the mining pipeline itself.

LLM adjudication (“do these perform the same purpose?”) for independent reimplementations. Pairwise edges are clustered into clone clusters—the unit of consolidation.

C. L3: Library Consolidation with Provenance

Clusters are ranked by member count, cross-project spread, and block size; cross-project spread is the strongest signal of a genuine shared capability. For each selected cluster, a canonical signature is designed to parameterize the members’ variations; the implementation is LLM-drafted and human-reviewed, packaged, and published to the private PyPI server. Provenance rows map each library function to every source block it subsumes, with notes on which parameter covers which variation. Replacement scripts—originals re-expressed as library calls—enter the catalog as new files linked to what they replace.

D. L4: Behavioral Validation

For each consolidated script, original outputs on representative inputs are captured as golden masters; original and replacement execute in isolated, dependency-pinned environments on identical inputs. Comparators are type-appropriate: byte equality for deterministic artifacts, tolerances for floats, schema-plus-content for dataframes, seeded statistical comparison for stochastic outputs. A script’s `retired` flag is set only when all its cases pass; failures return to L3 as defect reports.

E. L5: Generative Delivery with HITL Gate

Fig. 2 shows the request lifecycle. The generative model receives a customer request plus retrieval over the catalog and proposes a capability match and execution plan. Plans run as scheduled jobs recording exact library versions—extending provenance to the product level. Completed products enter a review queue carrying the request, plan, provenance chain, and outputs; the reviewer approves (scheduled delivery) or rejects (rework with notes). Below-threshold resolution or absent capability routes the request to a data scientist as a development ticket; delivered code onboards through L1–L4 and only then becomes resolvable capability. Scheduled periodic reports are standing requests on cron triggers using the identical machinery.

IV. PROVENANCE CHAIN

Fig. 3 traces a delivered product backward: delivery → approving review → job with pinned library versions → library functions → source blocks → files → projects. Every

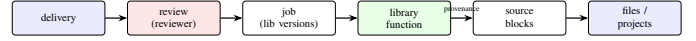


Fig. 3. Product-to-source provenance chain. Foreign keys at every hop.

link is a foreign key; the audit trail is a database query, not an investigation.

V. FAILURE MODES AND GUARDS

Failure	Guard
False clone merge / missed semantic clone	Three-tier detection; LLM adjudication and human review before consolidation
Library changes behavior silently	Golden-master gate; retirement only on full pass; defects route to L3
Lost audit trail	Mandatory provenance at function and product level
Unreviewed release	Schema-enforced: delivery requires approval row
Generative misresolution	Confidence threshold routes to development; reviewer sees plan and provenance
Corpus drift	Idempotent incremental re-index; new code enters the same pipeline

VI. DISCUSSION

The platform’s growth loop is its most important property: capability gaps do not bypass the architecture, they feed it. New code written for an unmet request is indexed, clone-checked, potentially consolidated, validated, and only then served—so the catalog’s guarantees hold uniformly over mined legacy code and freshly written code alike. The corpus stops being an archive and becomes the substrate of a delivery system whose every output is traceable, validated, and human-approved.